GravelKing Pro — Technical White Paper

GravelKing Pro Technical White Paper

Current-Code Architecture, Audio Processing, Provenance, and Legal Boundaries

Document status: Code-derived technical description
Platform: GravelKing Pro
Primary stack: TypeScript, Express 5, React, PostgreSQL/Drizzle, Clerk, Stripe, FFmpeg, Python/Numba, Vertex AI, ElevenLabs

---

1. Executive Technical Summary

GravelKing Pro combines:

- A creator-facing React application
- An Express API service
- PostgreSQL application records
- Stripe subscriptions and credit purchases
- Clerk identity with a legacy session-cookie fallback
- Local FFmpeg media handling
- A local Python Morris Law Kernel mastering worker
- Vertex AI Gemini for songwriting text
- ElevenLabs for text-to-speech and an alternate music-generation path
- A dual-anchor cryptographic certificate system
- PostgreSQL plus Firestore certificate backup

The current code uses the labels **MLK v3**, **MLK v3.5**, and **Morris Law Kernel v3.5** in different layers:

- `kernel-v3.ts` contains the JavaScript/FFmpeg three-band carve and LSB transport helpers.
- The primary mastering route invokes a local Python worker identified as MLK v3.5.
- API response headers identify mastered output as `MLK_v3.5`.

The Python worker is the sole mastering DSP path in the reviewed master route. FFmpeg is used for ingestion, trimming, denoising, normalization between formats, previews, conversion, and container/codec work.

---

2. System Architecture

2.1 Frontend

The web client is a React application with routes for:

- Songwriting/JAX
- Vocal Booth
- The Foundry/mastering
- Studio and mix workflows
- Format conversion
- Library/vault
- Pricing and account management
- Certificate verification
- Investor and administrative surfaces

Authentication is handled by Clerk on the modern path. A legacy `gk_session` cookie remains for anonymous checkout/session compatibility and reviewer flows.

2.2 API

The API uses Express 5 and exposes route groups for:

- Authentication
- JAX text, voice, and music generation
- Audio processing and mastering
- Studio mixing
- Format conversion
- Credit balances and purchases
- Stripe and Play Billing
- Tracks and private-vault access
- Certificate creation, retrieval, and verification
- Public white-paper metadata
- Administrative and investor functions

2.3 Data

Application records use PostgreSQL with Drizzle ORM.

Important data categories include:

- Users and subscription tiers

- Credit balances and transaction history
- Tracks and ownership records
- Purchased/private track access
- Certificate stubs
- Usage and export quotas
- Referral and payment records

Stripe mirror data is managed separately in Stripe-controlled database tables. Certificate stubs are backed up to Firestore as a second audit location.

2.4 Storage

Generated audio is divided between:

- Private, ownership-gated full assets
- Public preview assets
- Temporary local files used during processing

The JAX music route creates a private full track and a public 30-second preview, then records ownership in the database.

---

3. Morris Law Kernel Audio Architecture

3.1 Three-Band Carve

The shared MLK v3 implementation divides audio into:

- Low band
- Mid band
- High/detail band

The fast FFmpeg implementation uses:

- Low: low-pass below 250 Hz
- Mid: high-pass at 250 Hz and low-pass at 4 kHz
- High: high-pass above 4 kHz

Band multipliers are derived from the main intensity:

- Low = intensity × 1.15, capped at 2.0
- Mid = intensity
- High = intensity × 0.80, floored at 0.1

The bands are recombined and dynamically normalized.

The in-process Float32 implementation performs comparable three-band carving and adaptive peak normalization. It sets a normalized target peak of 0.92. The full-length hot path avoids

that in-memory implementation because allocating whole-song arrays can exhaust the shared Node process.

3.2 Primary Mastering Path

The current mastering route:

1. Accepts an uploaded source.
2. Normalizes supported input into WAV when needed.
3. Optionally trims a 30-second sample.
4. Optionally applies FFmpeg denoise.
5. Reads the pre-kernel bytes for certificate binding.
6. Calls the local Python `mlk_master.py` worker.
7. Applies the chosen kernel preset and controls.
8. Returns WAV or converts the mastered WAV to 320 kbps MP3.
9. Optionally creates and embeds a certificate payload.

There is no remote DSP fallback in this route. If the Python kernel fails, the request fails explicitly.

3.3 Mastering Controls

The route passes these controls into the Python kernel:

- Kernel preset ID
- Intensity
- Sidechain filter
- Sidechain frequency
- Stereo-link setting
- Adaptive mode
- Automatic threshold toggle
- Automatic-threshold offset
- Target LUFS
- Output ceiling

The route documentation assigns the Python kernel ownership of:

- EQ shelves
- Saturation
- Adaptive sidechain compression
- Brickwall limiting
- Loudness staging

FFmpeg denoise uses frequency-domain and non-local-means filters before the kernel when selected.

3.4 Preset Target Matrix

Public preset	Kernel preset	Target LUFS	Ceiling
Baseline	`natural_body`	−14	−0.8 dB
Spacious	`spatial_edge`	−14	−0.8 dB
Normal	`natural_body`	−16	−1.0 dB
Broadcast	`gravelking_max`	−23	−2.0 dB
Vinyl	`warm_vintage`	−16	−1.0 dB
Podcast	`natural_body`	−16	−1.5 dB
Club	`sub_fire`	−12	−0.5 dB
Film	`natural_body`	−24	−2.0 dB
YouTube	`natural_body`	−14	−1.0 dB
SoundCloud	`gravelking_max`	−11	−0.5 dB
Apple	`natural_body`	−16	−1.0 dB

Therefore, “−14 LUFS / −1 dBTP” is not the universal system target. It is the YouTube preset target. Baseline is −14 LUFS with a −0.8 dB ceiling.

The reviewed TypeScript route calls the value a `ceiling`. The exact true-peak measurement behavior must be validated inside the Python DSP and with measurement fixtures before marketing every value as dBTP.

3.5 Phase, Stereo, and Mid/Side Claims

The JavaScript MLK documentation describes “phase-coherent recombination,” and the Python route exposes a stereo-link control.

The reviewed code does **not** establish a universal claim that every master performs a dedicated mid/side polarity-alignment stage. That statement should not be used in technical or marketing material without direct evidence from the Python worker and numerical validation.

---

4. Vocal Booth and Studio Mix

4.1 Client Recording

Vocal Booth uses browser audio APIs to:

- Decode uploaded tracks
- Compute waveform peaks
- Display a scrolling timeline

- Monitor live microphone data
- Record vocals
- Apply monitoring/recording effects
- Align vocals with a guide track
- Mix and download results

Timed lyrics can come from synced lyric data. When precise timing is unavailable, approximate timing is used and should be labeled as such.

4.2 Server Mix

The server mix route:

- Requires the Studio/King entitlement
- Rejects unauthorized users before upload parsing
- Accepts up to six files
- Limits each file to 50 MB
- Applies rate and concurrency controls
- Normalizes tracks to 44.1 kHz stereo
- Supports sequential or layered arrangement
- Supports speed and pitch changes
- Supports light or heavy noise reduction
- Returns a PCM WAV
- Passes the final mix through the fast MLK v3 carve

MIDI is rejected because the server route does not include a soundfont synthesizer.

---

5. JAX Architecture

5.1 Songwriting Text

JAX is constrained to songwriting:

- Lyric drafting
- Sections
- Rhyme
- Meter
- Imagery
- Hooks
- Bridges
- Constructive lyric feedback

Non-music prompts are redirected back to songwriting.

The request can include:

- The current prompt
- Up to 12 recent conversation messages
- An artist-profile JSON object, truncated to 6,000 characters

The current text-provider order is:

1. Vertex AI Gemini
2. Replit Gemini proxy fallback

The configured model in both clients is Gemini 2.5 Flash. The Vertex client uses the `us-central1` Google Cloud endpoint.

5.2 JAX Access Controls

The current route includes:

- Sign-in enforcement
- Twelve text requests per minute
- Five free JAX prompts per day for ordinary accounts
- Unlimited text prompts for eligible monthly, developer, or Node Auditor accounts

The daily counters in this route are in-memory Maps. They reset if the API process restarts and are not a durable billing ledger.

5.3 ElevenLabs

ElevenLabs is used for:

- JAX text-to-speech playback
- Selectable male voice presets
- An alternate generated-music endpoint

The TTS route requests `mp3_44100_128` using the multilingual voice model. It is not implemented as a 16 kHz PCM WebSocket in the reviewed code.

The ElevenLabs music route:

- Accepts lyrics, style, and title
- Requests up to a two-minute generated take
- Costs 20 credits
- Refunds credits when provider generation fails
- Creates a private vault track
- Creates a public 30-second preview

5.4 Vertex/Lyria Generation

The MLK generation route uses a Vertex-backed generation pipeline, then stores the result in the creator’s vault.

It:

- Requires lyrics for lyric-mode generation
- Supports instrumental and random modes
- Screens user-supplied lyrics server-side
- Charges 20 credits
- Applies rate and global concurrency limits
- Refunds credits on failed generation

Although the route name contains `generate-master`, project policy now treats generation/remix output as unmastered and expects mastering to be a separate paid action. Any technical or UI path that still auto-masters generated audio should be treated as a consistency defect.

---

6. Human-Authorship and Edit Evidence

The songwriting client maintains an authorship ledger and uses the shared authorship library to calculate a 0–100 score.

The system is designed to preserve:

- Final text
- Revision activity
- Edit count
- Timestamps
- Content hashes
- Account identity
- Certificate signatures

This can support a showing that a signed-in account developed and revised a work over time.

It cannot establish with certainty:

- Who physically typed each character
- Whether collaborators consented
- Whether material was copied outside the tracked editor
- Whether the account owner is the legal author
- Whether a work is sufficiently original under applicable law

The authorship score is product evidence, not a judicial conclusion.

---

7. Credit and Entitlement Architecture

7.1 Canonical Credit Costs

Action	Credits
Generate a song	20
Remix a song	20
Master plus download	75
Stamp a certificate	0

7.2 Subscription Resets

Plan	Reset amount	Billing cadence
Pro	800 credits	Each paid week
King	2,500 credits	Each paid month

Subscription credits are reset to the plan amount. They do not roll over.

7.3 Credit Packs

Pack	Base	Bonus	Total	Price
Starter	500	250	750	$10
Artist	1,250	500	1,750	$20
Studio	2,500	1,000	3,500	$30

7.4 Ledger Safety

Credit spending:

- Runs in a database transaction
- Uses an atomic “balance is sufficient” update
- Stores a transaction reference
- Makes retries idempotent

Credit grants:

- Use stable references
- Ignore duplicate webhook delivery

Generation failures and certain downstream mastering failures grant compensating refunds.

7.5 Current Pricing

The frontend loads live Stripe prices and uses these fallback values:

- Pro: $6.99/week
- King: $24.99/month
- Node Auditor: $249.50/month

Live Stripe or Google Play catalog values can supersede the fallback display. Catalog parity should be monitored so stale products cannot silently change checkout behavior.

7.6 Export Limits

The pricing page currently advertises:

- Pro: 10 WAV exports per rolling week
- King: 40 WAV exports per rolling month
- Paid plans: unlimited MP3 exports

The converter server currently requires a paid tier even though the converter page includes stale “Free” copy.

---

8. Certificate and Chain-of-Custody Architecture

8.1 Certification Is Opt-In

The master route creates a certificate only when `certify=true`.

Without certification:

- No content hash is stored
- No LSB payload is embedded
- No certificate database row is created

This allows covers, karaoke tracks, reference mixes, and ordinary masters to be processed without automatically making an ownership assertion.

8.2 Source Binding

The certificate binds to the pre-kernel source bytes:

```
contentHash = SHA-256(pre-kernel audio bytes)
```

The system then creates:

```
fullHash = SHA-256(contentHash | artistHandle | certId)
nominator = first 32 hex characters of fullHash
denominator = final 32 hex characters of fullHash
handshake = HMAC-SHA256(secret, certId | nominator | denominator | optional industry IDs)
```

8.3 Anchor A — Track-Embedded Nominator

The nominator is stored in a compact JSON payload and embedded in 16-bit PCM sample least-significant bits.

Wire format:

- Five-byte magic marker
- Two-byte payload length
- Opaque JSON payload

The watermark:

- Survives lossless WAV/FLAC copies
- Does not depend on ordinary metadata tags
- Is expected to be destroyed by lossy MP3/AAC re-encoding

The certified WAV must therefore be retained as the archival verification copy.

8.4 Anchor B — Server-Retained Denominator

The denominator, content hash, handshake, artist, timestamp, attribution, ownership, and optional industry identifiers are stored in a server-side certificate stub.

The server record is required for full verification. Possession of the nominator alone is not sufficient.

8.5 Industry-Identifier Binding

Optional identifiers can include:

- IPI/CAE
- ISWC
- ISRC

When present, they are added to the HMAC payload. A post-certification change to those identifiers causes a handshake mismatch.

8.6 Dual Storage

The primary certificate record is stored in PostgreSQL.

A backup record is also sent to Firestore. This provides a separate audit location, although claims such as “independently subpoenable” are legal characterizations and should be reviewed by counsel.

8.7 Certificate Documents

The JSON certificate can contain:

- UTC generation timestamp
- Lyrics hash
- Style-prompt hash
- Raw PCM audio hash
- Content hash
- Nominator
- Denominator
- HMAC handshake
- Optional industry identifiers
- Generation model
- Certified category
- Provenance
- 0–100 style-authorship score
- Copyright-screen status and provider

The JSON record is the canonical structured evidence. The current PDF generator is a minimal human-readable wrapper and explicitly notes that a production-grade PDF renderer should replace it for finalized legal documents.

---

9. Verification Operations

9.1 `/api/kernel/verify-cert`

This public endpoint accepts a WAV upload and:

1. Extracts the LSB payload.
2. Parses the certificate ID, nominator, and artist.
3. Looks up the server-side certificate stub.
4. Recomputes the HMAC.
5. Reconstructs the source-bound hash split.
6. Returns a valid or invalid result.

Failure reasons include:

- No watermark
- Corrupt payload
- Certificate missing from server

- Handshake mismatch
- Nominator mismatch

9.2 `/api/kernel/verify-signal` and `/api/v1/verify`

The expanded public verifier returns:

- Anchor A status
- Anchor B status
- HMAC status
- Certificate ID
- Artist
- Certification timestamp
- Kernel version
- A warrant state

Warrant states:

- `INTACT`
- `TAMPERED`
- `NO_WATERMARK`

The expanded verifier supports both current industry-ID-bound handshakes and a legacy handshake format.

9.3 Certificate Access

Real certificate documents are:

- Owner-only
- Hidden from non-owners with a not-found response
- Available as JSON or PDF after unlock

The current effective logic treats certificate unlocks as free and unlimited. The codebase still contains $1.99 Stripe PaymentIntent and Checkout fallback endpoints, but the active status and unlock paths currently make the certificate available without payment.

This should be described as:

> Certification is currently free.

It should not be described as a reliably enforced $1.99 paywall unless the active unlock behavior is changed and re-verified.

---

10. Copyright Screening

The generation route performs a server-side check on user-supplied lyrics before invoking the generation provider.

The certificate schema can record:

- Screening status
- Provider
- Local-catalog or global-commercial scope
- Scan timestamp

A “no match” result means no match was detected within the provider and scope used. It is not a legal clearance opinion and cannot guarantee that the work is non-infringing.

The system should distinguish:

- Global commercial-catalog screening
- Local-signature screening
- Screening unavailable
- Screening not run

---

11. Legal Interpretation Under U.S. Copyright Law

11.1 What the System Can Support

The product can help preserve evidence relevant to:

- Proof of existence at a particular time
- Possession of a particular source version
- Revision history
- Human editing activity
- Audio-source hashes
- Account attribution
- Chain-of-custody integrity
- AI-model disclosure
- Rights-identification metadata

These records may be useful when documenting human contributions to AI-assisted work.

11.2 What the System Cannot Guarantee

The system does not itself determine:

- Copyrightability
- Originality

- Legal authorship
- Ownership
- Work-for-hire status
- Valid assignment
- Collaborator shares
- Non-infringement
- Court admissibility
- Registration with the U.S. Copyright Office

17 U.S.C. and Copyright Office policy require fact-specific analysis. A cryptographic timestamp proves data relationships and server verification; it does not convert unprotectable material into protectable authorship.

The phrase “court-admissible certificate” should therefore be treated as a product aspiration, not a guaranteed legal result. Admissibility depends on authentication, relevance, hearsay rules, expert testimony, system reliability, and the court.

11.3 Recommended Legal Positioning

Use:

> Timestamped cryptographic proof-of-existence and chain-of-custody record designed to support authorship and provenance review.

Avoid:

> Guaranteed ownership, guaranteed copyright, automatic legal protection, or guaranteed court acceptance.

---

12. Security and Integrity Controls

The reviewed code includes:

- Authentication before private operations
- Ownership checks for certificate documents
- Fail-closed behavior for legacy ownerless certificates
- Pre-upload entitlement checks on the studio-mix route
- Upload-size limits
- Audio-duration limits
- Rate limits
- Concurrency limits
- Temporary-file cleanup

- HMAC comparison
- Atomic credit spending
- Idempotent credit grants
- Private full-track storage
- Public previews separated from private originals
- Short-lived HMAC download links for large mastered WAV files

Security caveat

Several certificate functions fall back to a hardcoded string if `SESSION_SECRET` is missing. Production must always provide a strong `SESSION_SECRET`; otherwise HMAC security is weakened.

---

13. Current-Code Discrepancies Requiring Careful Documentation

The following differences are visible in the reviewed code:

1. **Pricing:** current fallback Pro pricing is $6.99/week, not $9.99.
2. **Mastering target:** loudness and ceiling are preset-specific.
3. **JAX transport:** reviewed voice playback is MP3 over HTTP, not a 16 kHz PCM WebSocket.
4. **JAX provider:** Gemini 2.5 Flash provides text; ElevenLabs provides voice and an alternate music engine.
5. **Converter access:** UI copy says “Free,” while the server requires Pro or King.
6. **Certificate payment:** $1.99 purchase endpoints exist, but current unlock logic is free and unlimited.
7. **Certificate privacy:** white-paper metadata claims certificate retrieval is unauthenticated, but the actual document routes enforce owner authentication.
8. **Mid/side claim:** no universal mid/side polarity-alignment stage was established by the reviewed TypeScript route.
9. **PDF status:** the JSON certificate is canonical; the PDF renderer is intentionally minimal.
10. **Legal effect:** the certificate supports evidence preservation but does not guarantee ownership, registration, admissibility, or non-infringement.

These should be resolved in code or reflected honestly in public copy before enterprise or legal marketing.

---

14. Conclusion

GravelKing Pro’s technical value is the connection between:

- Song development

- Performance recording
- Audio mastering
- Format delivery
- Provenance evidence

The strongest technical differentiator is not any single effect. It is the combined chain:

```
 creative activity
→ source material
→ finished audio
→ cryptographic source binding
→ server-retained verification record
→ owner-controlled certificate
```

When described accurately, the platform can offer creators and partners a useful production and evidence workflow without overstating what software alone can prove under copyright law.